Computational Physics Exercises

Week 09: Autodifferentiation in PyTorch

Bjørk Hammer
Department of Physics and Astronomy
Aarhus University
Denmark

Contents

W09A Formulating simple loss functions 2
W09B The gradient and a first steepest descent 3
W09C Using `torch.nn.Model` to fit a circle 7
W09D Gaussian fit 10
W09E Spring models and catenary 12
W09F Lennard-Jones-Gauss systems 15
W09G Maximizing entropy for double well potential 17
W09H Dynamic equilibrium for three-body orbit 20
W09I Derivatives of functions 25

© Bjørk Hammer

October 8, 2026

Exercise W09A Formulating simple loss functions

Linear fit

Make a number of (x_i, y_i) data points. Fit a straight line,

$$y = wx + b,$$

to the points using `torch.nn.Module`. Use this as the loss function:

$$\mathcal{L}oss = \frac{1}{N} \sum_i |y(x_i) - y_i|^2$$

where N is the number of data points.

Center of mass in 1D

Pick some random x-values between 0 and 2. Shift the values so their mean value is 5. Formulate a `torch.nn.Module` module with a gradient-aware parameter, x_{cm}, and a `forward` function that receives a vector of x-values and returns their distance to x_{cm}.

Use the module to optimize the $\mathcal{L}oss$-function:

$$\mathcal{L}oss = \frac{1}{N} \sum_i |x_i - x_{cm}|^2$$

Read out the `cm` parameter after the optimization. It must be 5.

Roots of function

Formulate a `torch.nn.Module` that when given a function upon instantiation can be used via optimization to give the roots of the function. Use this loss function:

$$\mathcal{L}oss = |f(x)|^2 .$$

Exercise W09B The gradient and a first steepest descent

In this exercise you will be using PyTorch to evaluate the gradient of a scalar function,

$$\nabla f(x_1, x_2, \ldots, x_N) = \left(\frac{\partial}{\partial x_1} f, \frac{\partial}{\partial x_2} f, \ldots \frac{\partial}{\partial x_N} f \right)$$

The scalar function

Make a contour plot for the function:

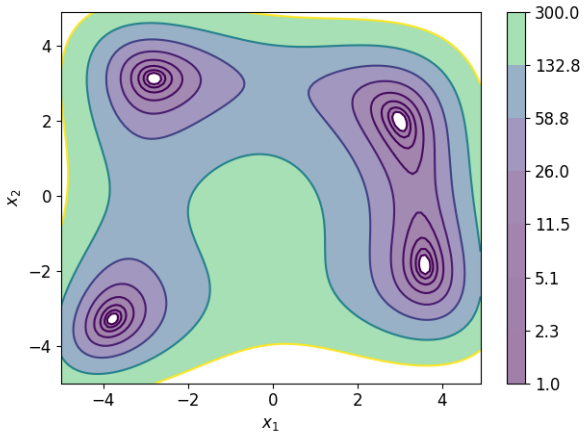
$$f(x_1, x_2) = \left(x_1^2 + x_2 - 11\right)^2 + \left(x_1 + x_2^2 - 7\right)^2$$

Hint: complete this code:

```
f = YOUR CODE

vals = YOUR CODE
x1d, x2d = np.meshgrid(vals,vals)
yd = f(x1d, x2d)

fig, ax = plt.subplots()
levels = YOUR CODE
ax.contour(x1d, x2d, yd, levels=levels)
color_map = ax.contourf(x1d, x2d, yd, levels=levels, alpha=0.5)
fig.colorbar(color_map, ax=ax)
ax.set(xlabel='$x_1$', ylabel='$x_2$')
```

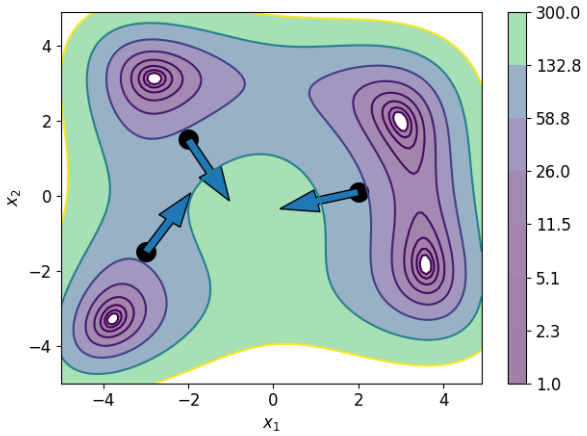


The gradients

Use pytorch to evaluate the gradients at some places in the contour plot. Plot the gradients as vectors. Check that they are perpendicular to the contour lines.

Hint: Complete this code:

```
for P0 in [[-2,1.5],[2,.1],[-3,-1.5]]:
    P0 = YOUR CODE
    YOUR CODE FOR FORWARD PASS
    YOUR CODE FOR BACKWARD PASS
    grad_of_y = YOUR CODE FOR GRADIENT EVALUATION
    P0 = P0.detach()
    v = grad_of_y/np.linalg.norm(grad_of_y)
    ax.scatter(*P0,s=200,c='k')
    ax.arrow(*P0,*v,width=0.2)
```



Optimizing manually: Steepest descent

Perform some steepest descent steps, where the gradient is calculated by automatic differentiation. Plot the trajectory.

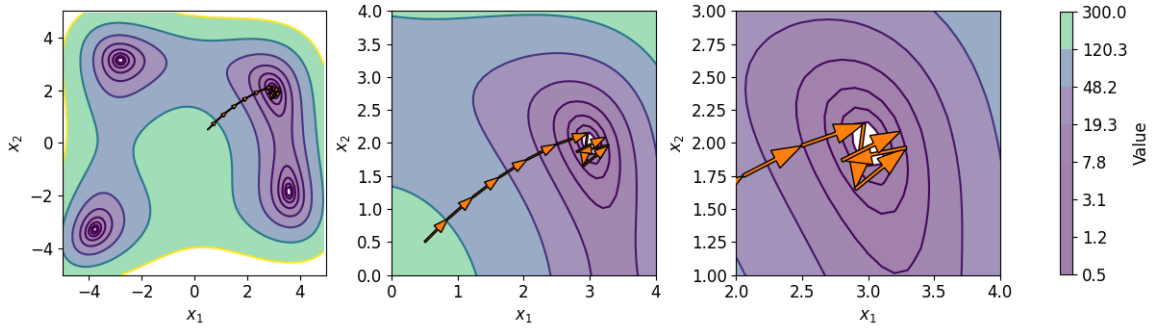
Now complete the code below, which should take you to a local minimum in the scalar function.

```
P0 = torch.tensor([ 0.5, 0.5], requires_grad=True)

REMEMBER_ZEROING_GRADIENT = True

for _ in range(10):
    if REMEMBER_ZEROING_GRADIENT and P0.grad is not None:
        P0.grad.zero_()
    y = # YOUR CODE forward calculation of function value
    y   # YOUR CODE backward calculation of gradient
    grad = P0.grad

    with torch.no_grad():
        v = grad / torch.norm(grad)
        step_size = 0.5
        P0 += YOUR CODE
        print(P0.data) # change this to some plotting directives
```



Try to set `REMEMBER_ZEROING_GRADIENT = False` and see what happens.

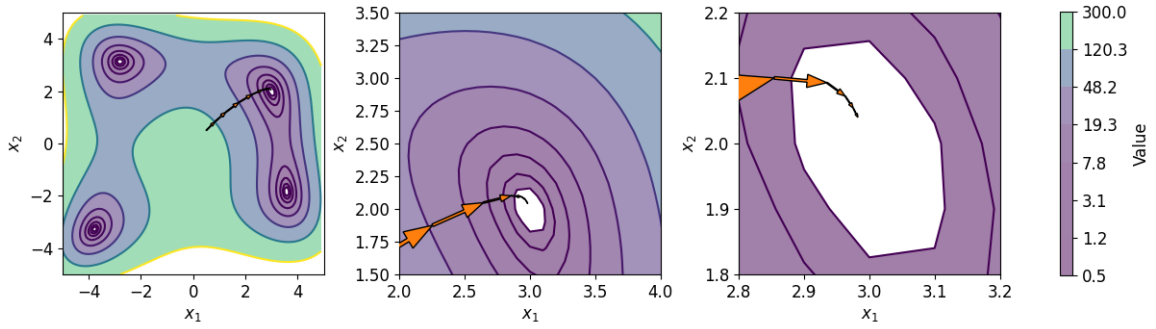
Gradient descent

Modify the above code so as do perform gradient descent.

Hint. The update now looks like:

```
P0 += -learning_rate * grad
```

where the gradient is not normalized and `learning_rate` is some small value.



Optimizing with `nn.Module`

Repeat the last task, but using the PyTorch standard module, `torch.nn.Module` together with a PyTorch optimizer.

Hint: Complete this code:

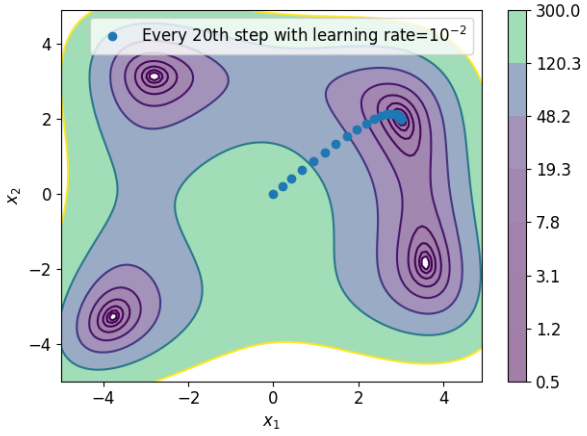
```
class ScalarField(torch.nn.Module):
    def __init__(self, f, P0 = [0.5, 0.5]):
        super().__init__()
        self.f = f
        self.P0 = torch.nn.Parameter( YOUR CODE
```

```
    def forward(self):
        y = YOUR CODE
        return y

YOUR CODE instantiating the class

optimizer = torch.optim.Adam( YOUR CODE

steps = 1000
for step in range(steps):
    optimizer.zero_grad()
    loss   YOUR CODE for forward calc of function value
    loss   YOUR CODE for backward calc of gradient
    optimizer.step()
```



Exercise W09C Using `torch.nn.Model` to fit a circle

This exercise is about practicing using the `torch.nn.Model`. Such models are very pratical whenever you perform optimizations. In this exercise, only a few parameters are optimized, but later in the course, the many parameters of neural networks will be optimized in the same way.

A parabola

Consider the parabola:

$$y(x) = ax^2 + bx + c$$

a) Setup a PyTorch model, `torch.nn.Model`, where a, b, and c are parameters. Use some arbitrary values for the parameters to begin with.

Fitting

b) Consider this data:

$$\begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}3\\4\end{bmatrix}, \begin{bmatrix}5\\1\end{bmatrix}$$

Now use your pytorch model to determine a, b, and c, so as to make the curve $(x, y(x))$ fit the data best possibly.

Hint 1: The `torch.optim.Adam` optimizer might work well with a 0.01 learning rate. The `torch.optim.SGD` might require a smaller learning rate.

Hint 2: You can write the mean-square-error loss function like this:

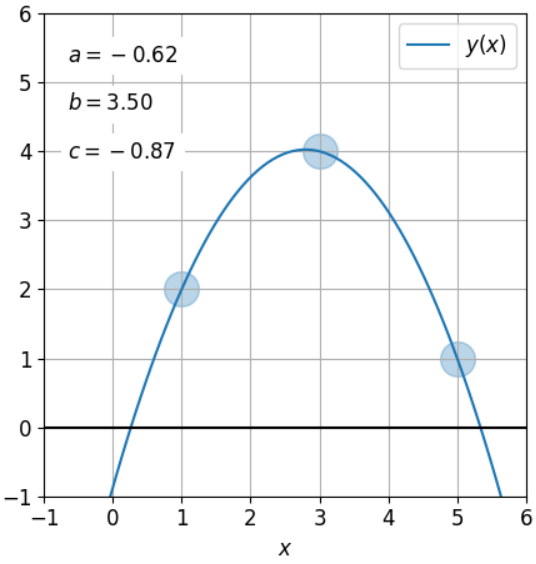
```
loss = torch.mean((y_prediction - y_train) ** 2)
```

Hint 3: After having handled the loss-function yourself, you might want to write it like this:

```
# outside training loop:
loss_func = torch.nn.MSELoss()

# inside training loop:
  loss = loss_func(y_prediction, y_train)
```

which will be faster for applications with much data.

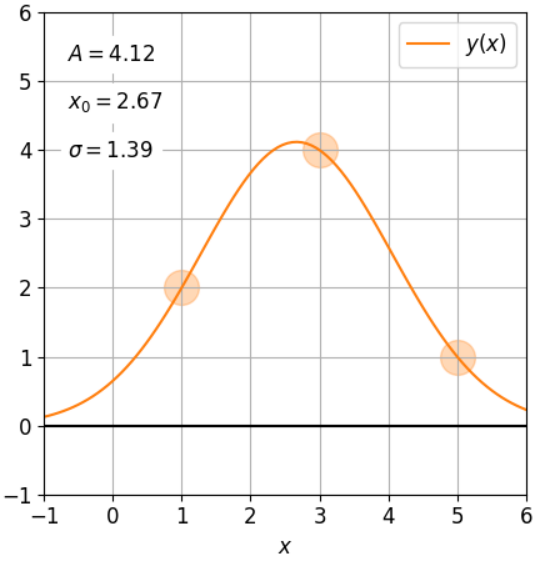


Gaussian

c) Fit instead a Gaussian,

$$f : f(x) = A \cdot \exp\left(-\frac{(x - x_0)^2}{2\sigma^2}\right)$$

to the data using the same technique as before.

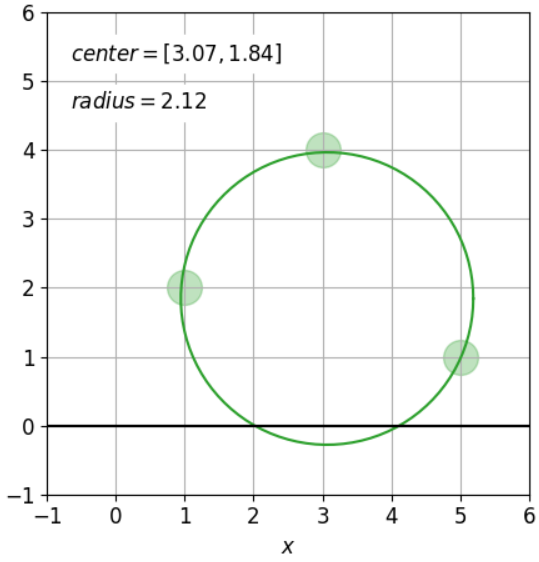


Circle

d) Now fit a circle through the three points.

Hint: The loss-function is the hard part. Here is what it should contain:

- Evaluate distances of each point to the current center.
- Evaluate the mean of the squared deviations of those distances to the current radius.



Exercise W09D Gaussian fit

a) Make a pytorch model for a Gaussian:

$$f : f(x) = A \cdot \exp\left(-\frac{(x - x_0)^2}{2\sigma^2}\right)$$

b) Assume $x_0 = -1$ and $\sigma = 0.3$. Fit the amplitude, A, to this data:

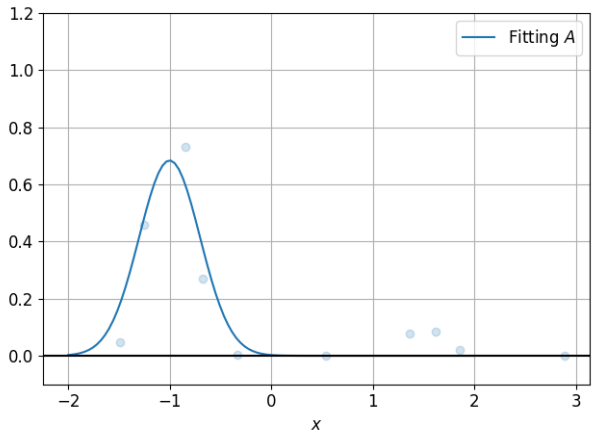
```
np.array([[-1.493,  0.048],
       [-1.25 ,  0.459],
       [-0.842,  0.731],
       [-0.676,  0.269],
       [-0.332,  0.004],
       [ 0.544,  0.   ],
       [ 1.363,  0.079],
       [ 1.621,  0.083],
       [ 1.853,  0.021],
       [ 2.892,  0.   ]])
```

Hint: Assume your model is called `gaussian` and has `A`, `x0`, and `sigma` as trainable parameters. Then this will train all parameters in the model:

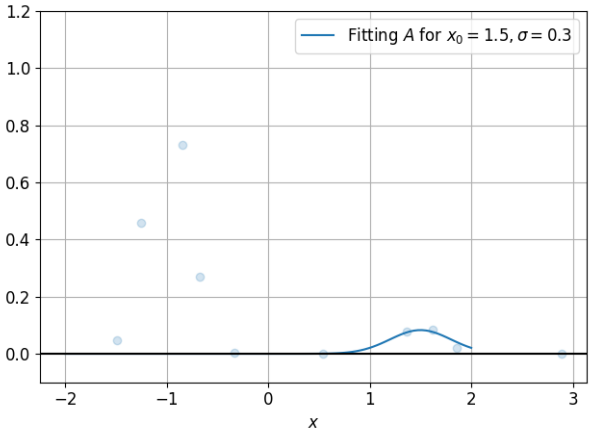
```
optimizer = torch.optim.SGD(gaussian.parameters(), lr=learning_rate)
```

while this will select only the `A`-parameter:

```
optimizer = torch.optim.SGD([gaussian.A], lr=learning_rate)
```

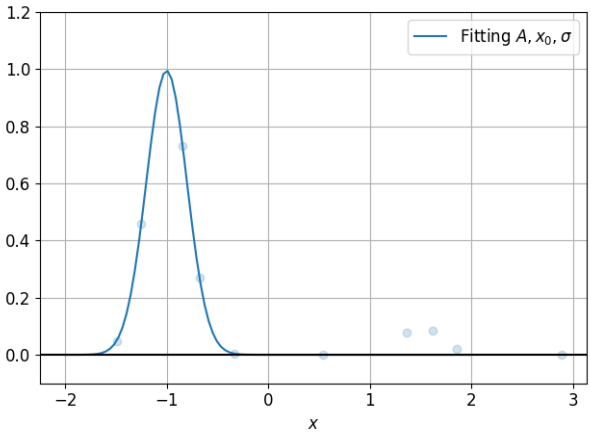


c) Assume $x_0 = 1.5$ and $\sigma = 0.3$. Fit again the amplitude, A.



Full fit

d) Now fit all three parameters, A, x_0, and σ. Play around with initializing the model with different (A, x_0, σ)-values and see what kind of fits you get.



Exercise W09E Spring models and catenary

In this exercise you will work with beads connected by springs. In the first exercise, you will optimize the position of one bead to minimize the energy of two springs. This can be programmed with just two ”optimizable” parameters, x and y, which can be held in two separate one-dimensional torch-tensors. In the next exercises you will optimize the positions of many beads, and some higher-dimensional tensor(s) with the positions will make the code look clearer.

A bead between two springs

a) Optimize the position of a central bead held by two springs attached to some fixed beads.

Hint: Go through these steps when plannning:

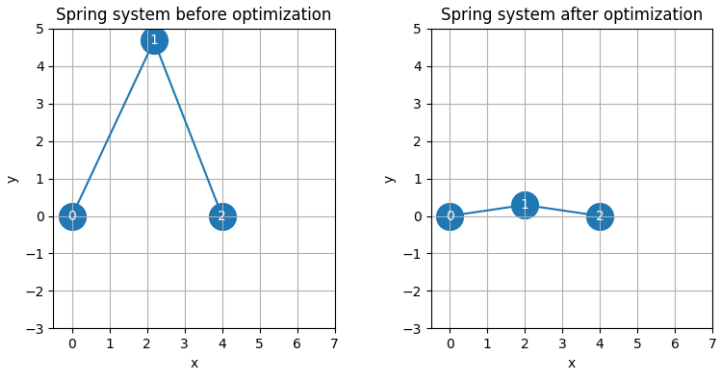
- Make a `torch.nn.Module`-model where the optimizable coordinates are held.
- Make an energy function as the `forward`-function of the model.
- Choose an optimizer that is aware of the optimizable coordinates.
- Optimize where the energy is the loss function, which is obtained by calling the model without parameters. I.e. if

  ```
  class SpringSystem(torch.nn.Module):
      ...
      def forward(self):
          ...
          return energy
  model = SpringSystem()
  ```

  then

  ```
  energy = model()
  ```

  triggers the call of the `forward`-function due to how `torch.nn.Module` is defined.
- Obtain and use for plotting the optimized positions.



A string of beads

b) Now generalize you solution to a) so as to handle a whole string of beads connected by springs.

Hint: Depending on how you implement it you may want to have a ”mask” that sets to zero the gradient on the end-beads. This is particularly the case if you keep the positions of all beads in one single torch tensor (which is convenient for the energy evaluation). The mask can be implemented something like this:

```
with torch.no_grad():
    model.YOUR-STATIC-PARAMETER.grad.zero_()
```

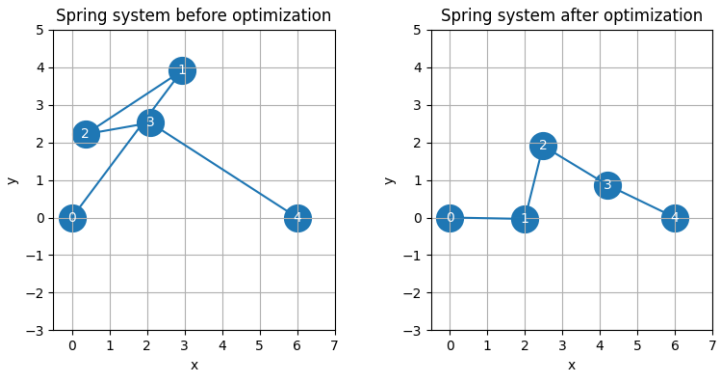
or

```
with torch.no_grad():
    model.YOUR-STATIC-PARAMETERS[indices].grad.zero_()
```

where `YOUR-STATIC-PARAMETER` or `YOUR-STATIC-PARAMETERS[indices]` single out the components of the bead-positions that should remain fixed.

The above statement should appear **after** the gradients are evaluated, and **before** the optimizer changes the parameters. So something like:

```
for step in range(steps):
    ...
    energy.backward() # now gradients are calculated
    YOUR CODE # here zero out gradients for static parameters
    optimizer.step() # now move only dynamic beads.
    ...
```

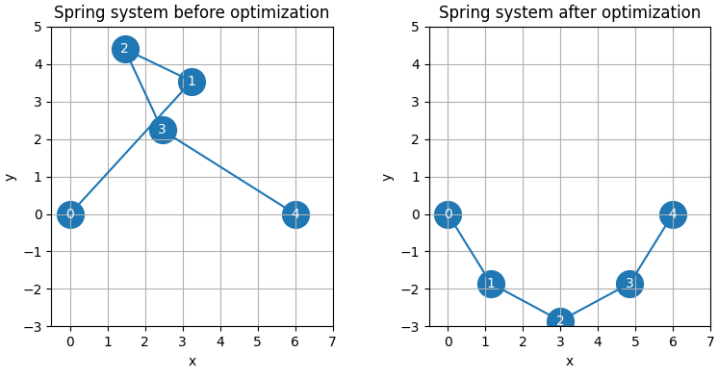


The catenary

c) Now add a gravitational force by extending the model class:

```
class SpringSystemWithGravity(SpringSystem):
    def forward(self):
        spring_energy = super().forward()
        gravitational_energy = YOUR CODE
        return spring_energy + gravitational_energy
```

Hint: You may have to try different learning rates and optimizers.



Exercise W09F Lennard-Jones-Gauss systems

The **Lennard-Jones-Gauss (LJG) potential** is an extension of the classic Lennard-Jones (LJ) potential. It incorporates a Gaussian term to account for more specific interactions between particles. This often leads to more complex structural patterns formed by the particles. The LJG potential models systems where particles have both short-range and longer-range attractions, and effectively, it encodes two preferred length-scales for the interactions. This means that patterns will form that show extra preferred particle separation distances.

The LJG potential takes the form:

$$V(r) = \left(\frac{1}{r}\right)^{12} - 2\left(\frac{1}{r}\right)^{6} - \varepsilon \cdot \exp\left(-\frac{(r - r_0)^2}{2\sigma^2}\right)$$

and is used to calculate total energies of atomic clusters by summing over all interatomic distances:

$$E = \sum_{i,j>i} V(r_{ij}).$$

Now, doing relaxation of atomic cluster described by the LJG-potential would ordinarily require us to derive the analytic derivate of $V(r)$ or estimate it with finite difference. However, automatic differentiation in PyTorch leverages such derivatives at almost no human effort, as will be seen in this exercise.

Setup classes

a) Start by introducing a calculator class:

```
class LennardJonesGauss:
    def __init__(self,eps=1, sigma_squared=0.02, r0=1.7):
        YOUR CODE

    def V(self,r):
        return YOUR CODE
```

and a `torch.nn.Module` class that will hold the atomic positions.

```
class LJGSystem(torch.nn.Module):
    def __init__(self, calc, Natoms):
        super().__init__()
        self.calc = calc
        self.positions = torch.nn.Parameter( YOUR CODE

    def forward(self):
        return YOUR CODE TO CALCULATE THE ENERGY

    def draw(self, ax):
        ax.scatter(self.positions.data[:, 0],
                   self.positions.data[:, 1], s=200)
```

Hint: There is a function `torch.pdist` that does the same as `scipy.spatial.distance.pdist`, but supports automatic differentiation.

b) To check your calculator, you can use these positions:

```
pos = torch.tensor([[4.065, 5.93 ],
                    [3.211, 5.465],
                    [5.711, 4.9  ],
                    [6.567, 5.374],
                    [4.113, 6.903],
                    [5.741, 5.879],
                    [5.662, 3.924],
                    [4.893, 5.411],
                    [4.844, 4.443],
                    [4.036, 4.953],
                    [6.539, 4.384],
                    [4.931, 6.382]])
```

and should expect -40.667.

Search

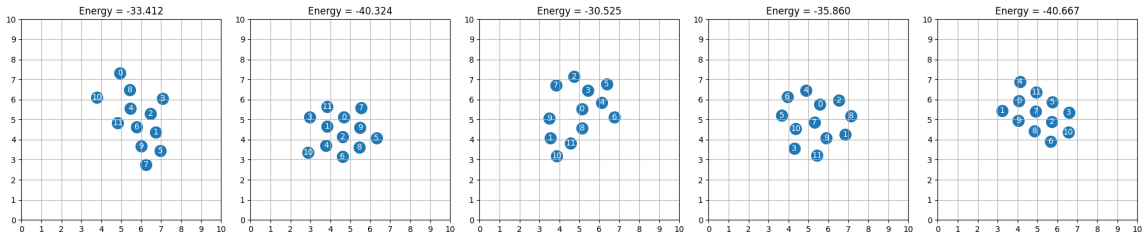
c) Now initiate the atoms at random positions, and perform optimization with respect to the atomic positions using the energy as the loss function.

Hint: If the distance between two randomly initialized atoms is very short, the gradient may become too large to handle for the optimizer. You can limit the allowed values of the calculated gradient by inserting something like this:

```
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=0.5)
```

right before the optimizer makes the step.

d) Do it a number of times and enjoy that you get relaxed structures without having had to derive a force-expression analytically.



Exercise W09G Maximizing entropy for double well potential

A one-dimensional potential

Implement this smooth one-dimensional potential consisting of two harmonic wells:

$$V(x) = -\frac{1}{\alpha}\ln\left(e^{-\alpha V_1(x)} + e^{-\alpha V_2(x)}\right),$$

where

$$V_1(x) = \tfrac{1}{2}k_1(x - x_1)^2, \qquad V_2(x) = \tfrac{1}{2}k_2(x - x_2)^2 + \Delta E.$$

Here, x_1 and x_2 denote the positions of the minima of the two harmonic wells, k_1 and k_2 their respective force constants, and ΔE is the energy offset of the second well relative to the first. The parameter α controls how sharply the potential transitions between the two wells: for large α, the potential closely follows the lower of the two parabolas at each x.

a) Implement the above potential and evaluate it using the following parameters: $x_1 = -2$, $x_2 = 2$, $k_{1x} = 8$, $k_{2x} = 8$, $\Delta E = 2$, and $\alpha = 10$.

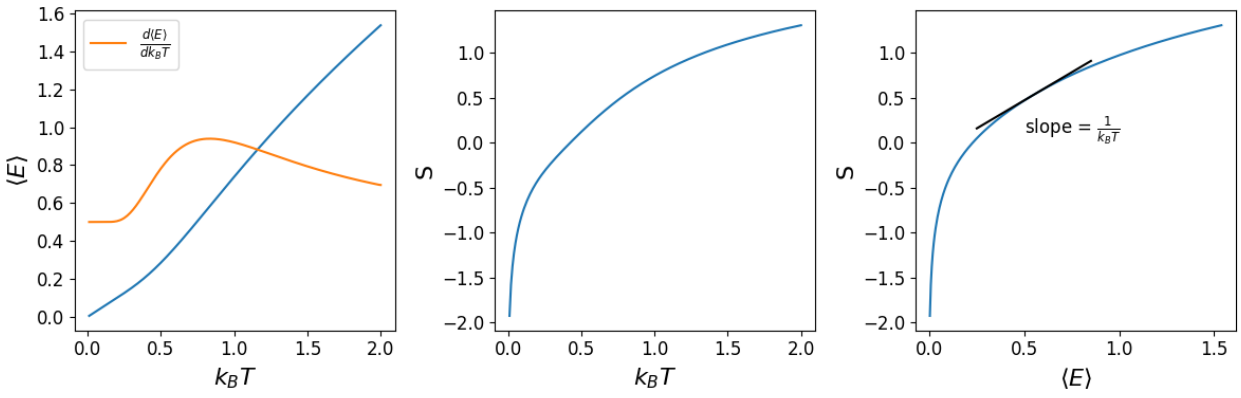
Results from direct integration

b) Evaluate by means of direct numerical integration the expected energy of the canonical ensemble of a particle in the double well when in contact with a heat bath at temperature, $k_BT = 0.8$. Integrations may be limited to ranging from -5 to 5. You should obtain: $\langle E\rangle = 0.5517$.

c) Compute the entropy of the particle. Since temperature is expressed as k_BT, the result corresponds to the dimensionless Shannon entropy, not the Gibbs entropy (which would have units of k_B). You should obtain: $S = 0.53618$.

d) Evaluate $\langle E\rangle$ and S as a function of temperature, k_BT. Plot S versus $\langle E\rangle$ and draw a line through $(\langle E(k_BT)\rangle, S(k_BT))$ with slope $\frac{1}{k_BT}$ (i.e. for $k_BT = 0.8$). Notice how it is a tangent, which makes probable the relation:

$$\frac{1}{k_BT} = \frac{dS}{d\langle E\rangle}$$



An analytic algebra task

e) Prove the above relation from first-principles (i.e. from expressions such as $S = -\int P(x) \ln P(x) dx$ and $P(x) = \frac{1}{Z} \int \exp\left(-E(x)/(k_B T)\right)\ dx$, etc.).

Variational solution

Recall that:

$$F = \langle E \rangle_{P(x)} - k_B T S[P(x)]$$

is stationary for the equilibrium distribution, $P_{eq}(x)$ and fixed $\langle E \rangle$. In fact, F is minimized since S is maximized in equilibrium. This means that if $P(x)$ is represented on a grid (as we have done whenever it has been plotted as a histogram), the values $P(x_i)$ can be varied as to minimize $S[P(x)]$ subject to the constraints that the expected energy is correct and that the probability distribution integrates to 1. The resulting loss function is:

$$\mathcal{L}oss = -S[P(x)] + \lambda_1 \left| \int P(x)\ dx - 1 \right|^2 + \lambda_2 \left| \langle E \rangle_{P(x)} - E_{target} \right|^2$$

where E_{target} is the desired value of $\langle E \rangle$.

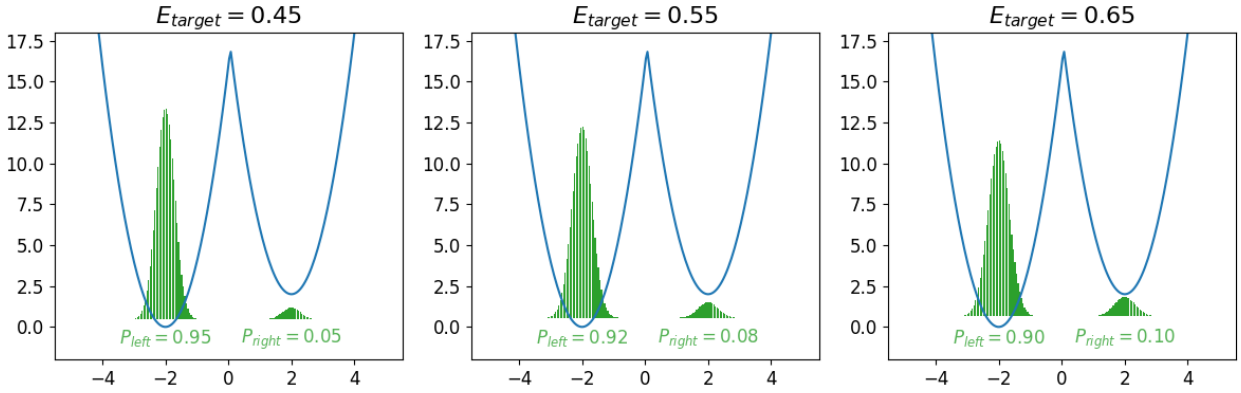
f) Implement a `torch.nn.Module` that holds the $P(x)$ as optimizable parameter. Optimize $P(x)$ according to the loss function using $E_{target} = 0.5517 \pm \delta E$, where δE is a variation. Plot the resulting $P(x)$.

Hint finish this code:

```
class MaxEntDistribution(torch.nn.Module):
    def __init__(self, xs, potential1D, expected_energy, lambda1=100.0,
   lambda2=100.0):
        super().__init__()
        self.delta_x = (xs[1]-xs[0])
        self.Vs = torch.tensor(potential1D(xs), dtype=torch.float64)
        self.E_target = expected_energy
        self.lambda1 = lambda1
        self.lambda2 = lambda2
        self.logits = torch.nn.Parameter(torch.ones_like(self.Vs))

    def forward(self):
        p = torch.nn.functional.softmax(self.logits, dim=0) # softmax
   guarantees that P(x) cannot be negative
        entropy = -torch.sum(p/ self.delta_x * torch.log(p/ self.delta_x + 1
   e-10)) * self.delta_x
        norm_constraint = (torch.sum(p) - 1.0)**2
        energy_constraint = YOUR CODE

        loss = -entropy + self.lambda1 * norm_constraint + self.lambda2 *
   energy_constraint
        return loss, p
```



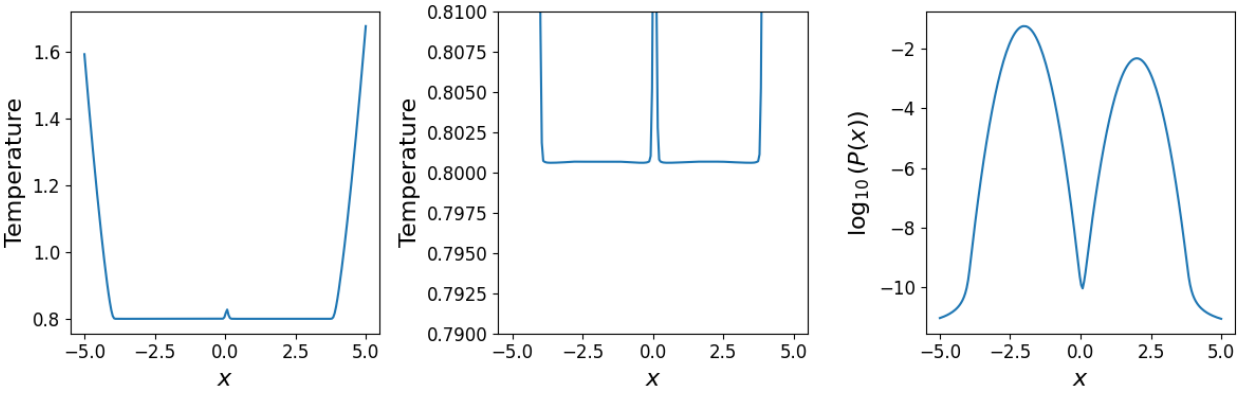
Sanity checks

g) Print out values of S, and $\langle E \rangle$. You might have to increase the values of the regularization parameters.

h) Since you used three different values of E_{target}, you may use finite difference to evaluate the slope of S with respect to E_{target}. Extract from that $k_B T$ and check its validity.

i) Choose $x_{ref} = x_1$ and plot as a function of x:

$$k_B T(x) = -\frac{V(x) - V(x_{ref})}{\ln\left(P(x)/P(x_{ref})\right)}$$



Yet an analytic algebra task

i) Derive the above equation.

Exercise W09H Dynamic equilibrium for three-body orbit

In this exercise, you will use `PyTorch` and automatic differentiation to optimize the initial conditions of a simple three-body system, such that the trajectories become periodic. The system uses a gravitational interaction in two dimensions.

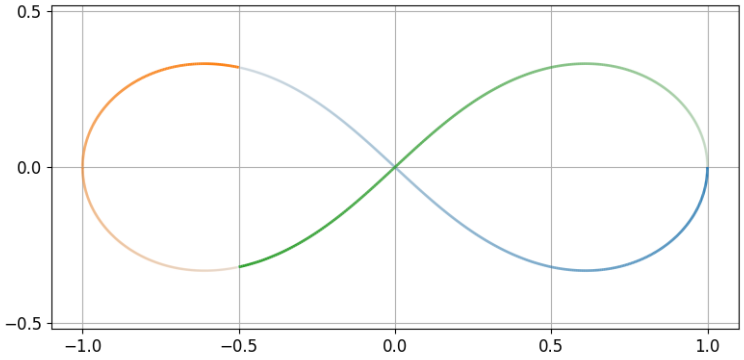
The initial setting is the following: Three particles with the same mass are started out at related positions:

$$\mathbf{r}_0 = \left(r_{0x}, r_{0y}\right), \qquad \mathbf{r}_1 = \left(r_{0x}, -r_{0y}\right), \qquad \mathbf{r}_2 = \left(-2r_{0x}, 0\right),$$

and velocities:

$$\mathbf{v}_0 = \left(v_{0x}, v_{0y}\right), \qquad \mathbf{v}_1 = \left(-v_{0x}, v_{0y}\right), \qquad \mathbf{v}_2 = \left(0, -2v_{0y}\right),$$

The optimizable parameters are r_{0x}, r_{0y}, v_{0x}, and v_{0y}. The objective is to conduct a fixed number of Velocity-Verlet molecular dynamics timesteps, $N_{steps} = 333$, with timestep $\Delta t = 0.00563433$ and adjust the optimizable parameters until the three particles follow the same trajectory, like this:



The positions

The class below sets up a symmetric three-body configuration and stores trainable parameters for the initial position and velocity for one of the bodies.

```
class ThreeBodyOrbit(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.r0 = torch.nn.Parameter(torch.tensor([-0.54, 0.33]))
        self.v0 = torch.nn.Parameter(torch.tensor([1.1, -0.22]))
```

In the `forward` method the initial positions and velocities for all bodies must be set up and integrated by the velocity-Verlet algorithm. Start by completing the code for the initial velocities in this code snippet:

```
def forward(self, N=333):
    pos0 = torch.stack([
```

```
        self.r0,
        torch.stack([self.r0[0], -self.r0[1]]),
        torch.stack([-2 * self.r0[0], torch.tensor(0.0)])
    ])
    vel0 = # YOUR CODE THIS TASK

    pos, vel = pos0, vel0
    dt = self.dt

    # YOUR CODE when implementing velocity-Verlet

    return pos, vel, pos0, vel0
```

Then call the `forward` method and print out `pos0` and `vel0`.

```
pos0, vel0
```

```
(tensor([[-0.5400,  0.3300],
        [-0.5400, -0.3300],
        [ 1.0800,  0.0000]], grad_fn=<StackBackward0>),
 tensor([[ 1.1000, -0.2200],
         [-1.1000, -0.2200],
         [ 0.0000,  0.4400]], grad_fn=<StackBackward0>))
```

The gravitational force

Next, implement the gravitational force between all particles in a function `compute_gravitational_forces(pos)`.

The function takes a tensor of shape $(N, 2)$ containing the positions of N particles and should return a tensor of the same shape containing the total forces on each particle.

The gravitational interaction between two particles i and j is given by

$$\mathbf{F}_{ij} = Gm \frac{\mathbf{r}_{ij}}{(\left|\mathbf{r}_{ij}\right|^2 + \varepsilon^2)^{3/2}},$$

where $G = 1$, $m = 1$, $\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i$ and a small ε prevents numerical instabilities for small separations.

Use the following template:

```
def compute_gravitational_forces(pos, G=1.0, eps=1e-7):
    N = pos.shape[0]
    force = torch.zeros_like(pos)
    # YOUR CODE
    return force
```

Test it to get:

```
compute_gravitational_forces(pos0)
```

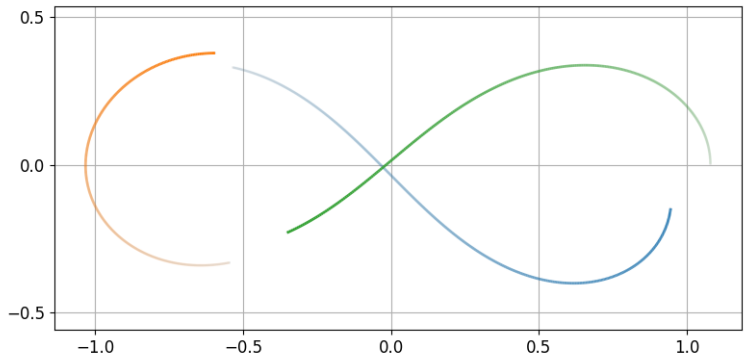
```
tensor([[ 0.3585, -2.3687],
        [ 0.3585,  2.3687],
        [-0.7170,  0.0000]], grad_fn=<CopySlices>)
```

The Velocity Verlet integration loop

Now complete the `forward` method by adding a Velocity Verlet integration loop

```
for _ in range(N):
    force = compute_gravitational_forces(pos)
    vel = vel +
    pos = pos +
    force_new =
    vel = vel +
```

Test the `forward` method by calling it once. Accumulate the trajectory somehow and plot it. You will get:



The loss function

Define a loss function that measures how close the final configuration is to the initial one, up to a cyclic permutation of the particles:

$$\mathcal{L}oss = \sum_i \left|\mathbf{r}_i^{(\text{final})} - \mathbf{r}_{i-1}^{(\text{initial})}\right|^2 + \sum_i \left|\mathbf{v}_i^{(\text{final})} - \mathbf{v}_{i-1}^{(\text{initial})}\right|^2 .$$

Use this template, where you have to permute the order of the final positions and velocities of the bodies:

```
def periodic_loss(model):
    pos_final, vel_final, pos0, vel0  = model()
    pos_final = pos_final[[ YOUR CODE], :]
    vel_final = vel_final[[ YOUR CODE], :]
    loss = torch.sum((pos_final - pos0)**2) + torch.sum((vel_final - vel0)
   **2)
    return loss
```

If everything is implemented correctly, you should obtain

$$L = 0.2437.$$

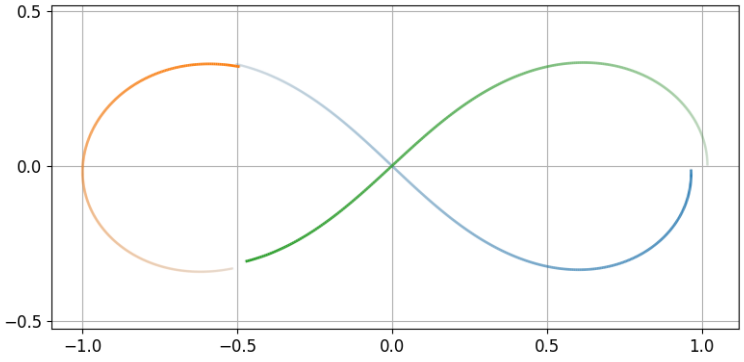
Optimizing the orbit

Finally, use `torch.optim.Adam` to optimize the parameters `r0`, and `v0` to minimize the loss.

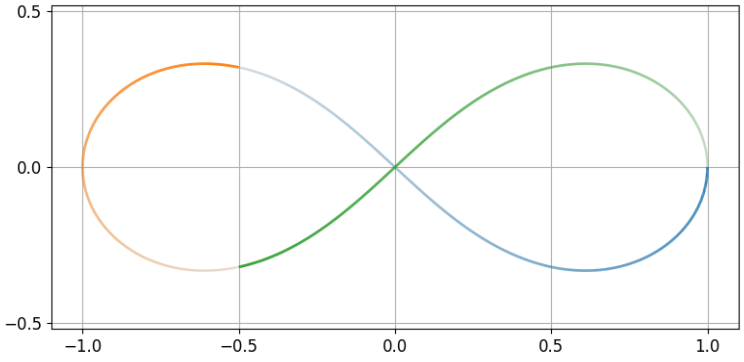
```
model = ThreeBodyOrbit()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for step in range(50):
    optimizer.zero_grad()
    loss = # YOUR CODE
    loss   # YOUR CODE
    optimizer.step()
    if step % 50 == 0:
        print(f"{step:4d}  Loss = {loss.item():.4f}")
```

After 50 optimization steps at learning rate 10^{-3} it should look like this:



And after about 500 steps you will get:



You will find that r_{0x} becomes 0.5.

Optimize the timestep

Modify your `nn.Module` so $r_{0x} = -0.5$ and is not optimizable. Instead, the timestep should be treated as a parameter. Here is a trick that makes sure that the timestep will always be positive (although it is probably not needed in the present context).

```
    def __init__(self, ...)
        # add this:
        self._raw_dt = torch.nn.Parameter(torch.tensor( YOUR VALUE

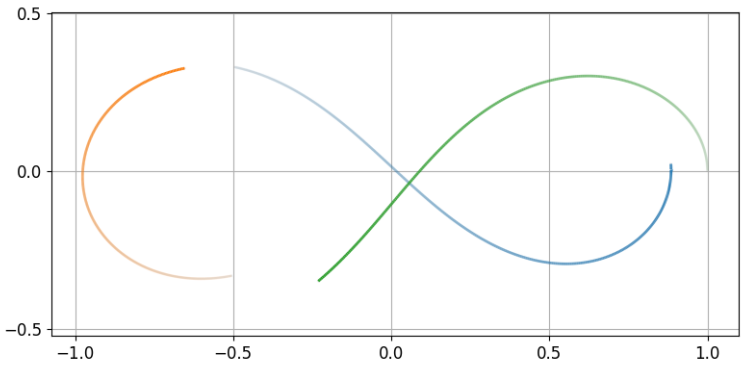
    @property
    def dt(self):
        return torch.nn.functional.softplus(self._raw_dt)
```

Note how a "hidden" trainable parameter, `self._raw_dt` is used to define the actual variable, `self.dt` providing the value for Δt. This is a commonly used trick to handle parameters that must remain positive. While the trainable parameter can have any value, positive or negative, while being optimized, the actual variable is computed using `torch.nn.functional.softplus`:

$$f_{\text{softplus}}(x) = \log\left(1 + \exp(x)\right)$$

and is therefore always positive.

Start from $\Delta t = 0.005$, work out what `self._raw_dt` must be to get that, and plot the trajectory without optimization of the parameters:



Now optimize the parameters (the r_{0y}, $\mathbf{v}_0$, and Δt). You should recover the periodic trajectories and the Δt used before.

Other stationary orbits

Look up the wikipedia website on the topic and get inspiration to find other periodic orbits.

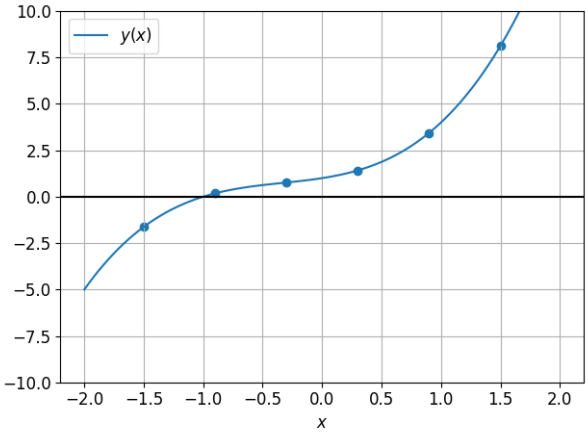
Exercise W09I Derivatives of functions

Polynomial

Consider the polynomial:

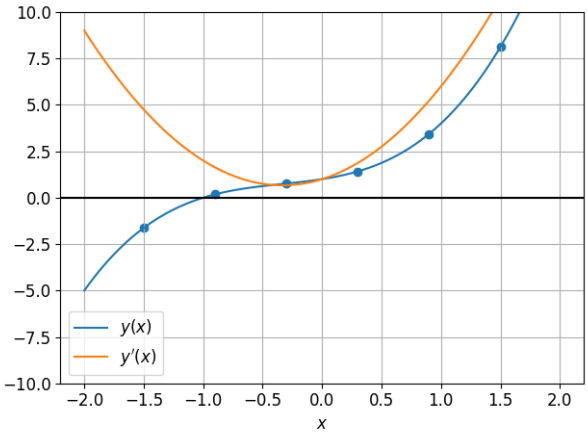
$$y(x) = ax^3 + bx^2 + cx + d$$

a) Choose some values for the coefficients, a, b, c, and d, and plot the function.



Automatic Differentiation

b) Use `pytorch` to obtain the first derivative of $y(x)$ and plot it.



Explanation for variable names in hints:

```
# x is an x-value
# xs are many x-values
# y is a y-value
```

```
# ys are many y-values
# yp is one y'-value ("p"=prime)
# yps are many y'-values
```

Hint 1: You can do it for one x-value at a time:

```
# one at a time
xs = torch.linspace(-2, 2, 100)
yps = []
for x in xs:
   x.requires_grad_(True)
   y =    YOUR CODE FOR THE FORWARD CALCULATION
   y.     YOUR CODE FOR THE BACKWARD CALCULATION
   yp =   YOUR CODE TO GET THE GRADIENT AT THIS x
   yps.append(yp.detach())
```

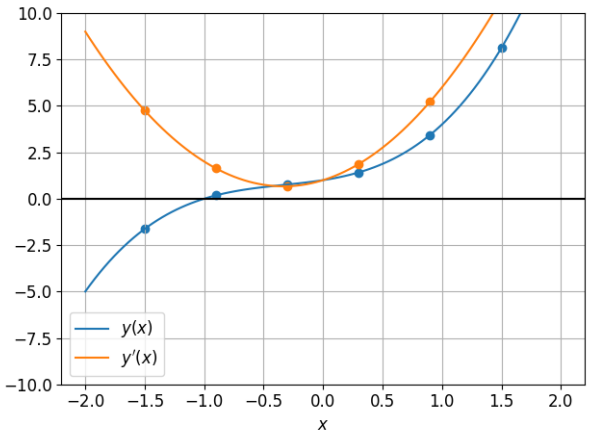
Hint 2: You can do it for all x-values at the same time:

```
# all at once
xs = torch.linspace(-2, 2, 100, requires_grad=True)
ys = YOUR CODE FOR GETTING ALL y-VALUES
yps = torch. YOUR CODE TO GET ALL GRADIENT IN ONE GO USING xs and ys

xs = xs.detach().numpy()
yps = yps.detach().numpy()
```

c) Plot for a few x-values the analytic result that you would get from evaluating the derivative yourself:

$$y'(x) = 3ax^2 + 2bx + c$$



Note that in the present context, the term ”gradient” would be a little misleading. What you are

finding is:

$$\begin{aligned}
\frac{d}{dx_1} y_1 &= f'(x_1) \\
\frac{d}{dx_2} y_2 &= f'(x_2) \\
&\vdots \\
\frac{d}{dx_N} y_N &= f'(x_N),
\end{aligned}$$

which is not a gradient. In fact, the term "per-sample-gradients" would be a better term if one considers $(x_i, y_i = f(x_i))$ as "samples".

Higher derivatives

d) Repeat b) and c) for the second derivative.

Hint: If you use `torch.autograd.grad` for the first derivative, you will need to add the argument `create_graph=True` to be able to make the second derivative:

```
yps = torch.autograd.grad(ys, xs, grad_outputs=torch.ones_like(xs),
    create_graph=True)[0]
```

